Rezumat - Claude

Ghid Complet FCPL Examen - Partea 1

Cuprins Partea 1

1. Piece of Code - Analiza și Optimizare
2. Client-Server Model și Threading
3. AST (Abstract Syntax Tree)
4. Semantical Actions on Grammar
5. Traducere High-Level → Low-Level

---

1. Piece of Code - Analiza și Optimizare

Exemplu: Optimizarea expresiilor

Expresie inițială: `y = (a - (20*b) - 25) * (c* (20-d) *25)`

i) Arborele Sintactic (AST)

```
        *

      /   \

     -     *

    / \   / \

   -  25 c   *

  / \       / \

 a   *    20   *

    / \       / \

  20  b     -    25

           / \

          20  d
```

ii) Cod Intermediar (Three-Address Code)

```
t1 = 20 * b

t2 = a - t1

t3 = t2 - 25

t4 = 20 - d

t5 = c * t4

t6 = t5 * 25

y = t3 * t6
```

iii) Cod Optimizat

```
// Calculăm constantele

t1 = 20 * b

t2 = a - t1 - 25  // Combinăm scăderile

t3 = 20 - d

t4 = c * t3 * 25  // t4 = c * (20-d) * 25

y = t2 * t4
```

iv) Cod Final (Assembly-like)

```
MOV R1, b

MUL R1, 20        ; R1 = 20 * b

MOV R2, a

SUB R2, R1        ; R2 = a - 20*b

SUB R2, 25        ; R2 = a - 20*b - 25
```

```
MOV R3, d

MOV R4, 20

SUB R4, R3        ; R4 = 20 - d

MOV R5, c

MUL R5, R4        ; R5 = c * (20-d)

MUL R5, 25        ; R5 = c * (20-d) * 25


MUL R2, R5        ; R2 = rezultat final

MOV y, R2
```

---

2. Client-Server Model și Threading

Model Client-Server

```
// SERVER - Multi-threaded

using System.Net;

using System.Net.Sockets;

using System.Threading;


public class Server {

    private TcpListener listener;

    public void Start() {

        listener = new TcpListener(IPAddress.Any, 8080);

        listener.Start();
```

```
        Console.WriteLine("Server pornit pe port 8080");
        while (true) {
            // Acceptă clienți
            TcpClient client = listener.AcceptTcpClient();
            // Creează un thread nou pentru fiecare client
            Thread clientThread = new Thread(() => HandleClient(client));
            clientThread.Start();
        }
    }
    private void HandleClient(TcpClient client) {
        NetworkStream stream = client.GetStream();
        byte[] buffer = new byte[1024];
        int bytesRead = stream.Read(buffer, 0, buffer.Length);
        string request = Encoding.ASCII.GetString(buffer, 0, bytesRead);
        Console.WriteLine($"Request: {request}");
        // Procesează și trimite răspuns
        string response = "HTTP/1.1 200 OK\r\n\r\nHello from server!";
        byte[] responseBytes = Encoding.ASCII.GetBytes(response);
        stream.Write(responseBytes, 0, responseBytes.Length);
        client.Close();
    }
}

// CLIENT
public class Client {
```

```
    public void Connect() {
        TcpClient client = new TcpClient("localhost", 8080);
        NetworkStream stream = client.GetStream();
        // Trimite request
        string request = "GET / HTTP/1.1";
        byte[] data = Encoding.ASCII.GetBytes(request);
        stream.Write(data, 0, data.Length);
        // Primește răspuns
        byte[] buffer = new byte[1024];
        int bytes = stream.Read(buffer, 0, buffer.Length);
        string response = Encoding.ASCII.GetString(buffer, 0, bytes);
        Console.WriteLine($"Response: {response}");
        client.Close();
    }
}
```

Cum funcționează Thread-urile

```
using System.Threading;

// PROBLEMĂ: Race Condition
public class BankAccount {
    private int balance = 1000;
    // GREȘIT - Nu este thread-safe
    public void WithdrawUnsafe(int amount) {
        if (balance >= amount) {
```

```
            Thread.Sleep(10); // Simulăm timp de procesare
            balance -= amount;
            Console.WriteLine($"Retras: {amount}, Balanță: {balance}");
        }
    }
    // CORECT - Thread-safe cu lock
    private readonly object _lock = new object();
    public void WithdrawSafe(int amount) {
        lock(_lock) { // Doar un thread poate intra aici
            if (balance >= amount) {
                Thread.Sleep(10);
                balance -= amount;
                Console.WriteLine($"Retras: {amount}, Balanță: {balance}");
            }
        }
    }
}

// Test
public class Program {
    static void Main() {
        BankAccount account = new BankAccount();
        // Creăm 2 thread-uri care încearcă să retragă simultan
        Thread t1 = new Thread(() => account.WithdrawSafe(600));
        Thread t2 = new Thread(() => account.WithdrawSafe(600));
```

```
        t1.Start();

        t2.Start();

        t1.Join(); // Așteaptă finalizarea

        t2.Join();

    }

}
```

Concepte Cheie Threading

Race Condition: Două thread-uri modifică aceeași dată simultan

Mutex/Lock: Mecanism de sincronizare - doar un thread în zona critică

Deadlock: Două thread-uri așteaptă unul după celălalt (blocare reciprocă)

---

3. AST (Abstract Syntax Tree)

Exemplu 1: `while (x > 0) { x = x - 1; }`

```
      while

     /     \

    >       =

   / \     / \

  x   0   x   -

             / \

            x   1
```

Exemplu 2: `if (a == b) c = 10; else c = 20;`

```

        if-else
```

```
    /   |   \
   ==   =    =
  /  \ / \  / \
 a   b c 10 c 20
```

Exemplu 3: `y = (a + b) * (c - d)`

```
    =
   / \
  y   *
     / \
    +   -
   / \ / \
  a  b c  d
```

Exemplu 4: Expresie complexă `z = a * b + c / d - e`

```
    =
   / \
  z   -
     / \
    +   e
   / \
  *   /
 / \ / \
```

```
  a  b c  d
```

Reguli pentru construcție AST:

- Operatorii cu prioritate mare sunt mai jos în arbore
- Ordinea de evaluare: de jos în sus, stânga la dreapta
- Frunzele sunt operanzi (variabile, constante)
- Nodurile interne sunt operatori

---

4. Semantical Actions on Grammar

Gramatică pentru expresii aritmetice

```
E -> E + T | E - T | T

T -> T * F | T / F | F

F -> (E) | num
```

Acțiuni Semantice (Evaluare)

```
E -> E1 + T    { E.val = E1.val + T.val }

E -> E1 - T    { E.val = E1.val - T.val }

E -> T         { E.val = T.val }


T -> T1 * F    { T.val = T1.val * F.val }

T -> T1 / F    { T.val = T1.val / F.val }

T -> F         { T.val = F.val }


F -> (E)       { F.val = E.val }
```

```
F -> num       { F.val = num.value }
```

Exemplu: Evaluează `3 + 5 * 2`

```
Parsare conform prioritității (* are prioritate):

T -> F -> 3              (val = 3)

T -> T * F -> 5 * 2      (val = 10)

E -> T + T -> 3 + 10     (val = 13)



Rezultat: 13
```

Gramatică Booleană

```
E -> E or T | T

T -> T and F | F

F -> true | false | (E)
```

Evaluare `true or false and true`:

```
and are prioritate mai mare:

T -> F and F -> false and true = false

E -> T or T -> true or false = true



Rezultat: true
```

Gramatică pentru expresii (din examen)

```
E -> E + T | T

T -> T * F | F

F -> (E) | ct_num
```

Evaluare `2 + 3 * 4`**:**

```
T -> T * F -> 3 * 4 = 12

E -> T + T -> 2 + 12 = 14



Rezultat: 14
```

---

5. Traducere High-Level → Low-Level

IF-ELSE Statement

High-Level (C):

```c
if (a > b)

    max = a;

else

    max = b;
```

Low-Level (Assembly):

```
    MOV R1, a

    MOV R2, b

    CMP R1, R2        ; Compară a cu b
```

```
    JLE ELSE_LABEL     ; Jump if Less or Equal (a <= b)

TRUE_LABEL:

    MOV max, R1        ; max = a

    JMP END_LABEL

ELSE_LABEL:

    MOV max, R2        ; max = b

END_LABEL:

    NOP
```

WHILE Loop

High-Level:

```
while (a > 0) {

    b = b + 1;

    a = a - 1;

}
```

Low-Level:

```
LOOP_START:

    MOV R1, a

    CMP R1, 0

    JLE LOOP_END       ; Dacă a <= 0, ieși din loop

    MOV R2, b

    ADD R2, 1

    MOV b, R2          ; b = b + 1
```

```
    SUB R1, 1

    MOV a, R1          ; a = a - 1

    JMP LOOP_START

LOOP_END:

    NOP
```

FOR Loop

High-Level:

```
for (i = 0; i < 10; i++) {

    sum = sum + i;

}
```

Low-Level:

```
    MOV i, 0           ; i = 0

FOR_LOOP:

    MOV R1, i

    CMP R1, 10

    JGE FOR_END        ; Dacă i >= 10, ieși

    MOV R2, sum

    ADD R2, R1

    MOV sum, R2        ; sum = sum + i

    MOV R1, i

    ADD R1, 1

    MOV i, R1          ; i++
```

```
    JMP FOR_LOOP
FOR_END:
    NOP
```

WHILE imbricat (din examen)

Cod:

```
a = 2025;
while (a > 0) {
    b = 2025;
    while (b > 0) {
        a--;
        b--;
    }
}
```

Low-Level:

```
    MOV a, 2025
OUTER_LOOP:
    CMP a, 0
    JLE OUTER_END
    MOV b, 2025
INNER_LOOP:
    CMP b, 0
    JLE INNER_END
```

```
    DEC a          ; a--

    DEC b          ; b--

    JMP INNER_LOOP

INNER_END:

    JMP OUTER_LOOP

OUTER_END:

    NOP
```

Set de Instrucțiuni Assembly

| Instrucțiune | Descriere |

|---|---|

| `MOV dest, src` | Copie valoare |

| `ADD dest, src` | Adunare |

| `SUB dest, src` | Scădere |

| `MUL reg` | Înmulțire |

| `DIV reg` | Împărțire |

| `INC reg` | Incrementare |

| `DEC reg` | Decrementare |

| `CMP op1, op2` | Comparare (setează flags) |

| `JMP label` | Salt necondiționat |

| `JE label` | Jump if Equal |

| `JNE label` | Jump if Not Equal |

| `JG label` | Jump if Greater |

| `JGE label` | Jump if Greater or Equal |

| `JL label` | Jump if Less |

| `JLE label` | Jump if Less or Equal |

| `NOP` | No operation |

Ghid Complet FCPL Examen - Partea 2

Cuprins Partea 2

6. Weakest Precondition & Postcondition
7. Faze de Compilare
8. Typing Systems - Comparație Limbaje
9. OOP Concepts & Implementation
10. Design Patterns

---

6. Weakest Precondition & Postcondition

Reguli de bază

Atribuire: `wp(x = E, Q) = Q[x/E]` (Înlocuiești x cu E în Q)

Secvență: `wp(S1; S2, Q) = wp(S1, wp(S2, Q))` (Calculezi de la coadă)

Notație Hoare Logic: `{P} S {Q}` unde:

- P = Precondiție
- S = Cod
- Q = Postcondiție

Exemplu 1: Atribuire simplă

Cod: `x = x + 10;`

Postcondiție: `{x > 20}`

Calcul WP:

```
Q: x > 20

Înlocuim x cu (x + 10):

(x + 10) > 20

x > 10
```

Weakest Precondition: `{x > 10}`

Exemplu 2: Secvență

Cod:

```
y = 5;

x = y + 2;
```

Postcondiție: `{x > 0}`

Calcul (de la coadă):

```
Step 1: wp(x = y + 2, {x > 0})

Înlocuim x cu (y + 2):

y + 2 > 0

y > -2


Step 2: wp(y = 5, {y > -2})

Înlocuim y cu 5:

5 > -2 = True
```

Weakest Precondition: `{True}`

Exemplu 3: IF Statement

Cod:

```
if (x > 0)

    y = x;

else
```

```
    y = -x;
```

Postcondiție: `{y > 0}`

Calcul:

```
Pentru ramura TRUE: wp(y = x, {y > 0}) = {x > 0}

Pentru ramura FALSE: wp(y = -x, {y > 0}) = {-x > 0} = {x < 0}



wp(if, Q) = (conditie ∧ wp_true) ∨ (¬conditie ∧ wp_false)

         = (x > 0 ∧ x > 0) ∨ (x ≤ 0 ∧ x < 0)

         = (x > 0) ∨ (x < 0)

         = x ≠ 0
```

Weakest Precondition: `{x ≠ 0}`

Exemplu 4: WHILE Loop

Cod:

```
while (x > 0) {

    x = x - 1;

}
```

Postcondiție: `{x == 0}`

Pentru loop-uri folosim invariant:

```
Invariant: I = {x >= 0}



Verificăm:
```

```
1. Intrarea: {x >= 0} este true înainte de loop

2. Conservare: {x >= 0 ∧ x > 0} x = x - 1 {x >= 0}

   Dacă x >= 1, atunci x - 1 >= 0 ✓

3. Terminare: {x >= 0 ∧ ¬(x > 0)} → {x == 0} ✓
```

Exemplu 5: Din examen - Transmitere prin referință

Cod:

```
void p2(int &x, int &y) {
    x = y * y;
    y = x * x;
}

void p1(int &x, int &y) {
    p2(y, x);
    x = y * y;
    y = x * x;
}

void main() {
    tab[0] = 3;
    tab[1] = 4;
    p1(tab[0], tab[1]);
}
```

Analiză pas cu pas:

```
Inițial: tab[0] = 3, tab[1] = 4


Apel p1(tab[0], tab[1]) → p1(x=3, y=4)


În p1:
  Apel p2(y, x) → p2(x=4, y=3):  // Atenție: y devine x, x devine y!
    x = y * y = 3 * 3 = 9
    y = x * x = 9 * 9 = 81
  După p2: y = 9, x = 81  // x și y din p1 acum
  x = y * y = 9 * 9 = 81
  y = x * x = 81 * 81 = 6561


Final: tab[0] = 81, tab[1] = 6561
```

Răspuns: `tab[0] = 81, tab[1] = 6561` (transmitere prin referință `&`)

---

7. Faze de Compilare

Toate fazele pentru: `int a = b + 5;`

1. Analiza Lexicală (Lexical Analysis / Scanning)

Input: `int a = b + 5;`

Output (Tokeni):

```
<KEYWORD, int>
<IDENTIFIER, a>
<ASSIGN, =>
```

```
<IDENTIFIER, b>

<OPERATOR, +>

<LITERAL, 5>

<SEMICOLON, ;>
```

Ce face: Împarte codul în "cuvinte" (tokens)

2. Analiza Sintactică (Parsing)

Verifică sintaxa și construiește AST:

```
    Declaration

       /    \

     int     =

            / \

           a   +

              / \

             b   5
```

Ce face: Verifică că ordinea tokenilor respectă gramatica

3. Analiza Semantică

Verificări:

- `b` este declarat? ✓
- Tipul lui `b` este compatibil cu `int`? ✓
- `a` este declarat în scope? ✓
- Conversii de tip necesare? ✓

Ce face: Verifică că codul "are sens"

4. Cod Intermediar (IR - Intermediate Representation)

```
t1 = b + 5

a = t1
```

Ce face: Generează cod independent de platformă (Three-Address Code)

5. Optimizare

```
// Dacă b este constantă (b = 10):

t1 = 10 + 5    // Constant folding

t1 = 15

a = 15



// Sau elimină variabile temporare inutile:

a = b + 5      // Direct assignment
```

Ce face: Îmbunătățește performanța (elimină cod mort, simplifică expresii)

6. Generare Cod Final (Code Generation)

```
MOV EAX, [b]      ; Încarcă b în registru

ADD EAX, 5        ; Adună 5

MOV [a], EAX      ; Salvează în a
```

Ce face: Traduce în assembly specific arhitecturii (x86, ARM, etc.)

Exemplu 2: `y = (x * 2) + 5;`

```
1. LEXICAL:

<ID,y> <ASSIGN,=> <LPAREN,(> <ID,x> <OP,*> <LIT,2> <RPAREN,)> <OP,+> <LIT,5>
```

```
<SEMI,;>


2. SYNTAX (AST):

      =

     / \

    y   +

       / \

      *   5

     / \

    x   2


3. SEMANTIC:

- x declarat? ✓

- Tipuri compatibile? ✓


4. IR:

t1 = x * 2

t2 = t1 + 5

y = t2


5. OPTIMIZARE:

t1 = x * 2

y = t1 + 5  // Eliminăm t2


6. COD FINAL:
```

```
MOV EAX, [x]

MUL EAX, 2

ADD EAX, 5

MOV [y], EAX
```

---

8. Typing Systems - Comparație Limbaje

Static vs Dynamic Typing

| Limbaj | Static/Dynamic | Strong/Weak | Caracteristici |

|--------|----------------|-------------|----------------|

| **C** | Static | Weak | Conversii implicite periculoase |

| **C++** | Static | Weak | Ca C + OOP |

| **Java** | Static | Strong | Tipare stricte, verificare runtime |

| **C#** | Static | Strong | Tipare stricte + type inference |

| **Python** | Dynamic | Strong | Tipare flexibile, erori la runtime |

| **JavaScript** | Dynamic | Weak | Conversii automate ciudate |

Definiții

Static Typing:

- Tipurile sunt verificate la **compilare**
- Trebuie să declari tipul: `int x = 5;`
- **Pro:** Siguranță, performanță, erori găsite devreme
- **Contra:** Verbozitate, mai puțin flexibil

Dynamic Typing:

- Tipurile sunt verificate la **execuție**
- Variabila ia tipul valorii: `x = 5` apoi `x = "text"`
- **Pro:** Flexibilitate, rapiditate în dezvoltare
- **Contra:** Erori doar la runtime

Strong Typing:

- Nu permite conversii implicite ciudate
- `"1" + 2` dă eroare
- **Exemplu:** Python, Java, C#

Weak Typing:

- Încearcă să convertească automat
- `"1" + 2` devine `"12"` (JS) sau `3` (PHP)
- **Exemplu:** C, JavaScript

Exemple Practice

C - Static & Weak

```
int x = 5;

float y = x;        // Conversie implicită OK

char c = 'A';

int z = c + 1;      // OK, 'A' devine 65, z = 66



// Periculos:

int* ptr = 100;     // Permite conversii nesigure
```

C++ - Static & Weak

```
int x = 5;

double y = x;       // Conversie implicită



// Mai sigur cu casting explicit:

double z = static_cast<double>(x);
```

Java - Static & Strong

```java
int x = 5;
String s = "Hello";

// String result = s + x;  // OK în Java (conversie specială)
// int y = s;  // EROARE: incompatibilitate de tip

// Conversie explicită necesară:
int num = Integer.parseInt("123");
```

C# - Static & Strong

```csharp
int x = 5;
string s = "Hello";

// string result = s + x;  // OK: "Hello5"
// int y = s;  // EROARE: nu se poate converti implicit

// Conversie explicită:
int num = int.Parse("123");
int? nullable = null;  // Nullable types
```

Python - Dynamic & Strong

```python
x = 5          # x este int
x = "text"     # Acum x este string - OK
```

```python
# y = "5" + 2  # EROARE: TypeError
y = "5" + str(2)  # OK: "52"
y = int("5") + 2  # OK: 7


# Duck typing:
def add(a, b):
    return a + b  # Funcționează pentru int, float, string, etc.
```

JavaScript - Dynamic & Weak

```javascript
let x = 5;
x = "text";    // OK


// Conversii ciudate:
let y = "5" + 2;    // "52" - concatenare string
let z = "5" - 2;    // 3 - conversie la număr
let a = "5" * "2";  // 10 - conversie la număr
let b = [] + [];    // "" - string gol
let c = [] + {};    // "[object Object]"


// Comparații ciudate:
console.log(0 == "0");     // true
console.log(0 === "0");    // false (strict equality)
console.log(null == undefined);  // true
```

Type Inference (C#)

```
// C# are type inference cu var

var x = 5;           // Compilatorul deduce int

var name = "John";   // Compilatorul deduce string

var list = new List<int>();  // Compilatorul deduce List<int>



// Dar e tot STATIC typing:

// x = "text";  // EROARE: x este int
```

---

9. OOP Concepts & Implementation

Concepte Fundamentale

1. **Encapsulation** - Ascunderea detaliilor de implementare
2. **Inheritance** - Moștenirea proprietăților de la clasă părinte
3. **Polymorphism** - Obiecte diferite răspund la aceeași interfață
4. **Abstraction** - Clase/metode abstracte care definesc comportament

Hierarchy de Clase - Problema Motorcycle (din examen)

```
// Clasă abstractă de bază

public abstract class Motorcycle {

    // Proprietăți - Encapsulation

    public string Brand { get; protected set; }

    public string Model { get; protected set; }

    private int _nominalSpeed;

    public int NominalSpeed {

        get { return _nominalSpeed; }
```

```
        protected set { _nominalSpeed = value; }
    }
    // Constructor
    protected Motorcycle(string brand, string model, int nominalSpeed) {
        Brand = brand;
        Model = model;
        NominalSpeed = nominalSpeed;
    }
    // Metodă abstractă - trebuie implementată de derivate
    public abstract void Run();
    // Metodă virtuală - poate fi suprascrisă
    public virtual string ToString() {
        return $"Brand: {Brand}, Model: {Model}, Speed: {NominalSpeed}";
    }
}


// Clasă derivată 1 - Inheritance
public class CrossMotorcycle : Motorcycle {
    public int ShockAbsorbers { get; protected set; }
    public CrossMotorcycle(string brand, string model, int speed, int
shockAbsorbers)
        : base(brand, model, speed) {  // Apelăm constructorul părinte
        ShockAbsorbers = shockAbsorbers;
    }
    // Implementare metodă abstractă
```

```
    public override void Run() {
        Console.WriteLine("Hello, I am CrossMotorcycle");
    }
    // Suprascriere metodă virtuală
    public override string ToString() {
        return base.ToString() + $", ShockAbsorbers: {ShockAbsorbers}";
    }
}


// Clasă derivată 2
public class SpeedMotorcycle : Motorcycle {
    public int TopSpeed { get; protected set; }
    public SpeedMotorcycle(string brand, string model, int speed, int
topSpeed)
        : base(brand, model, speed) {
        TopSpeed = topSpeed;
    }
    public override void Run() {
        Console.WriteLine("Hello, I am SpeedMotorcycle");
    }
    public override string ToString() {
        return base.ToString() + $", TopSpeed: {TopSpeed}";
    }
}


// Clasă container (Composition)
```

```
public class Garage {

    private List<Motorcycle> motorcycles;

    public Garage() {

        motorcycles = new List<Motorcycle>();

    }

    public void AddMotorcycle(Motorcycle m) {

        motorcycles.Add(m);

    }

    // Polymorphism - apelăm Run() pe tipuri diferite

    public void DisplayAll() {

        foreach (var m in motorcycles) {

            Console.WriteLine(m.ToString());

            m.Run();

        }

    }

}



// Utilizare - Polymorphism

public class Program {

    static void Main() {

        Garage garage = new Garage();

        // Adăugăm obiecte de tipuri diferite

        garage.AddMotorcycle(new CrossMotorcycle("Honda", "CRF450", 120,
4));

        garage.AddMotorcycle(new SpeedMotorcycle("Yamaha", "R1", 180, 300));
```

```
        // Polymorphism - fiecare răspunde diferit la Run()
        garage.DisplayAll();
        // Output:
        // Brand: Honda, Model: CRF450, Speed: 120, ShockAbsorbers: 4
        // Hello, I am CrossMotorcycle
        // Brand: Yamaha, Model: R1, Speed: 180, TopSpeed: 300
        // Hello, I am SpeedMotorcycle
    }
}
```

Problema Car (din examen - varianta 2)

```
public abstract class Car {
    public string Brand { get; protected set; }
    public string Engine { get; protected set; }
    public int Capacity { get; protected set; }
    public int YearOfManufacture { get; protected set; }
    protected Car(string brand, string engine, int capacity, int year) {
        Brand = brand;
        Engine = engine;
        Capacity = capacity;
        YearOfManufacture = year;
    }
    public abstract void Run();
    public virtual string ToString() {
        return $"{Brand} {Engine}, {Capacity}cc, Year: {YearOfManufacture}";
```

```
    }

}



public class RaceCar : Car {

    public double Acceleration0_100 { get; protected set; }

    public RaceCar(string brand, string engine, int capacity, int year,
double accel)

        : base(brand, engine, capacity, year) {

        Acceleration0_100 = accel;

    }

    public override void Run() {

        Console.WriteLine("Hello, I am RaceCar");

    }

    public override string ToString() {

        return base.ToString() + $", 0-100: {Acceleration0_100}s";

    }

}



public class GarbageTruck : Car {

    public int Load { get; protected set; }

    public GarbageTruck(string brand, string engine, int capacity, int year,
int load)

        : base(brand, engine, capacity, year) {

        Load = load;

    }

    public override void Run() {
```

```
        Console.WriteLine("Hello, I am GarbageTruck");
    }
    public override string ToString() {
        return base.ToString() + $", Load: {Load}kg";
    }
}


public class Garage {
    private List<Car> cars = new List<Car>();
    public void AddCar(Car car) {
        cars.Add(car);
    }
    public void ShowAll() {
        foreach (var car in cars) {
            Console.WriteLine(car.ToString());
            car.Run();
        }
    }
}
```

---

10. Design Patterns

1. Singleton Pattern

Scop: O singură instanță globală a clasei

```
public class Database {
    // Instanță statică privată
    private static Database _instance;
    private static readonly object _lock = new object();
    // Constructor privat - nimeni nu poate face new Database()
    private Database() {
        Console.WriteLine("Database initialized");
    }
    // Metodă publică de acces (thread-safe)
    public static Database Instance {
        get {
            lock(_lock) {  // Thread-safe
                if (_instance == null) {
                    _instance = new Database();
                }
                return _instance;
            }
        }
    }
    public void Query(string sql) {
        Console.WriteLine($"Executing: {sql}");
    }
}
```

```
// Utilizare

Database db1 = Database.Instance;

Database db2 = Database.Instance;

// db1 == db2 (aceeași instanță)
```

2. Factory Method Pattern

Scop: Crearea obiectelor fără a specifica clasa exactă

```
// Interfață produs

public interface ITransport {

    void Deliver();

}



// Produse concrete

public class Truck : ITransport {

    public void Deliver() {

        Console.WriteLine("Livrare cu camion pe șosea");

    }

}



public class Ship : ITransport {

    public void Deliver() {

        Console.WriteLine("Livrare cu vapor pe mare");

    }

}
```

```
public class Plane : ITransport {

    public void Deliver() {

        Console.WriteLine("Livrare cu avion prin aer");

    }

}



// Factory

public class TransportFactory {

    public static ITransport CreateTransport(string type) {

        switch(type.ToLower()) {

            case "road":

                return new Truck();

            case "sea":

                return new Ship();

            case "air":

                return new Plane();

            default:

                throw new ArgumentException($"Unknown transport type:
{type}");

        }

    }

}



// Utilizare

ITransport transport = TransportFactory.CreateTransport("road");
```

```
transport.Deliver();  // "Livrare cu camion pe șosea"
```

3. Observer Pattern

Scop: Notificarea automată a obiectelor când se schimbă starea

```
// Interfață Observer
public interface IObserver {
    void Update(string message);
}

// Subject (Observable)
public class NewsAgency {
    private List<IObserver> subscribers = new List<IObserver>();
    private string latestNews;
    public void Subscribe(IObserver observer) {
        subscribers.Add(observer);
    }
    public void Unsubscribe(IObserver observer) {
        subscribers.Remove(observer);
    }
    public void PublishNews(string news) {
        latestNews = news;
        NotifyAll();
    }
    private void NotifyAll() {
        foreach (var subscriber in subscribers) {
```

```
            subscriber.Update(latestNews);
        }
    }
}

// Concrete Observers
public class NewsChannel : IObserver {
    private string name;
    public NewsChannel(string name) {
        this.name = name;
    }
    public void Update(string message) {
        Console.WriteLine($"{name} received: {message}");
    }
}

// Utilizare
NewsAgency agency = new NewsAgency();
agency.Subscribe(new NewsChannel("CNN"));
agency.Subscribe(new NewsChannel("BBC"));

agency.PublishNews("Breaking: Important event!");
// Output:
// CNN received: Breaking: Important event!
```

```
// BBC received: Breaking: Important event!
```

Ghid Complet FCPL Examen - Partea 3 (Final)

Cuprins Partea 3

11. Exception Handling
12. Lambda Calculus - Reducții
13. List Comprehension
14. Memory Efficiency & Data Structures
15. Rezumat Final și Sfaturi pentru Examen

---

11. Exception Handling

Trigger Exceptions (Aruncă Excepții)

```
// 1. Throw standard exceptions
public double Divide(int a, int b) {
    if (b == 0) {
        throw new DivideByZeroException("Cannot divide by zero!");
    }
    return (double)a / b;
}

// 2. Custom exceptions
public class InvalidAgeException : Exception {
    public InvalidAgeException(string message) : base(message) { }
}
```

```
public class Person {
    private int age;
    public void SetAge(int value) {
        if (value < 0 || value > 150) {
            throw new InvalidAgeException($"Age {value} is invalid!");
        }
        age = value;
    }
}

// 3. Throw cu InnerException (Exception chaining)
public void ProcessData(string data) {
    try {
        int value = int.Parse(data);
        // Procesează...
    }
    catch (FormatException ex) {
        throw new InvalidOperationException("Failed to process data", ex);
    }
}

// 4. ArgumentException
public void SetName(string name) {
    if (string.IsNullOrEmpty(name)) {
        throw new ArgumentException("Name cannot be null or empty",
```

```
nameof(name));

    }

}
```

Handle Exceptions (Gestionează Excepții)

```
public class ExceptionHandling {

    public static void Main() {

        // 1. Try-Catch simplu

        try {

            int result = Divide(10, 0);

        }

        catch (DivideByZeroException ex) {

            Console.WriteLine($"Error: {ex.Message}");

        }

        // 2. Multiple catch blocks (ordinea contează!)

        try {

            string input = Console.ReadLine();

            int number = int.Parse(input);

            int result = 100 / number;

        }

        catch (FormatException ex) {

            Console.WriteLine("Invalid number format!");

        }

        catch (DivideByZeroException ex) {

            Console.WriteLine("Cannot divide by zero!");
```

```
}
catch (Exception ex) {
    // Catch-all pentru orice altă excepție
    Console.WriteLine($"Unexpected error: {ex.Message}");
}
finally {
    // Se execută ÎNTOTDEAUNA
    Console.WriteLine("Cleanup code - always executed");
}
// 3. Try-Catch-Finally cu resurse
FileStream file = null;
try {
    file = File.Open("data.txt", FileMode.Open);
    // Lucrează cu fișierul
    byte[] buffer = new byte[1024];
    file.Read(buffer, 0, buffer.Length);
}
catch (FileNotFoundException ex) {
    Console.WriteLine("File not found!");
}
catch (IOException ex) {
    Console.WriteLine($"IO Error: {ex.Message}");
}
finally {
    // Închide întotdeauna fișierul
```

```
        file?.Close();
    }
    // 4. Using statement (auto-cleanup) - RECOMANDAT
    try {
        using (FileStream fs = File.Open("data.txt", FileMode.Open)) {
            // Lucrează cu fișierul
        } // Aici se apelează automat Dispose()
    }
    catch (Exception ex) {
        Console.WriteLine($"Error: {ex.Message}");
    }
    // 5. Re-throw exception
    try {
        ProcessData();
    }
    catch (Exception ex) {
        Console.WriteLine("Logging error...");
        throw; // Re-aruncă excepția mai departe
    }
    // 6. When clause (C# 6+)
    try {
        int result = GetValue();
    }
    catch (Exception ex) when (ex.Message.Contains("network")) {
        Console.WriteLine("Network error occurred");
```

```
        }
        catch (Exception ex) {
            Console.WriteLine("Other error");
        }
    }
}
```

Exception Best Practices

```
// ✅ BUN - Specific și informativ
public void ProcessFile(string filename) {
    if (filename == null) {
        throw new ArgumentNullException(nameof(filename));
    }
    if (!File.Exists(filename)) {
        throw new FileNotFoundException($"File not found: {filename}");
    }
}

// ❌ RĂU - Generic și neutil
public void ProcessFile(string filename) {
    if (filename == null) {
        throw new Exception("Error");
    }
}
```

```
// ✅ BUN - Catch specific
try {
    // cod
}
catch (FileNotFoundException ex) {
    // Handle specific
}
catch (IOException ex) {
    // Handle specific
}

// ❌ RĂU - Catch prea general
try {
    // cod
}
catch (Exception ex) {
    // Prinde tot, masca erori
}
```

---

12. Lambda Calculus - Reducții

Notație și Reguli

Notație:

- `λx. M` - funcție cu parametru `x` și corp `M`
- `(λx. M) N` - aplicare funcție: funcția `λx. M` aplicată pe argument `N`

Reguli:

1. **β-reduction (Beta):** `(λx. M) N → M[x := N]` (înlocuiește x cu N în M)
2. **α-conversion (Alpha):** Redenumire variabile bound pentru a evita conflicte

Exemplu 1: Simplu

`(λx. x + 1) 5`

Reducere:

```
Înlocuim x cu 5 în corpul (x + 1):

5 + 1 = 6
```

Rezultat: `6`

Exemplu 2: Funcție care returnează funcție

`(λx. λy. x + y) 3`

Reducere:

```
Step 1: Aplicăm λx cu 3

Înlocuim x cu 3 în (λy. x + y):

λy. 3 + y



Rezultat: λy. 3 + y
```

Dacă aplicăm și pe 4:

`(λx. λy. x + y) 3 4`

```
Step 1: λy. 3 + y (de mai sus)

Step 2: Aplicăm λy cu 4

Înlocuim y cu 4:
```

```
3 + 4 = 7
```

Rezultat: `7`

Exemplu 3: Din examen

`E1 = (λg. (g λy.y) λb. (b b))`

Reducere pas cu pas:

```
Step 1: Aplicăm g cu (λy.y)

(λy.y) λb.(b b)


Step 2: Aplicăm λy.y cu argumentul λb.(b b)

Înlocuim y cu λb.(b b) în corpul y:

λb.(b b)
```

Rezultat: `λb.(b b)`

Exemplu 4: Din examen - Mai complex

`E2 = (λx. (x (λg. (g λy.y) λb. (b b))) λa.(a a))`

Reducere:

```
Step 1: Aplicăm x cu (λg. (g λy.y) λb. (b b))

(λa.(a a)) (λg. (g λy.y) λb. (b b))


Step 2: Aplicăm λa.(a a) cu argumentul

Înlocuim a cu (λg. (g λy.y) λb. (b b)):

(λg. (g λy.y) λb. (b b)) (λg. (g λy.y) λb. (b b))


Step 3: Din E1 știm că (λg. (g λy.y) λb. (b b)) → λb.(b b)
```

```
Deci avem:

(λb.(b b)) (λb.(b b))



Step 4: Aplicăm λb.(b b) pe λb.(b b)

Înlocuim b cu λb.(b b):

(λb.(b b)) (λb.(b b))
```

Rezultat: Buclă infinită (omega combinator) - `(λb.(b b)) (λb.(b b))`

Exemplu 5: Compus cu variabile multiple

`(λx. λy. λz. x + y + z) 1 2 3`

Reducere:

```
Step 1: (λx. λy. λz. x + y + z) 1

→ λy. λz. 1 + y + z



Step 2: (λy. λz. 1 + y + z) 2

→ λz. 1 + 2 + z

→ λz. 3 + z



Step 3: (λz. 3 + z) 3

→ 3 + 3 = 6
```

Rezultat: `6`

Exemplu 6: Church Numerals

```
0 = λf. λx. x
```

```
1 = λf. λx. f x

2 = λf. λx. f (f x)

3 = λf. λx. f (f (f x))


SUCC = λn. λf. λx. f (n f x)


SUCC 1:

(λn. λf. λx. f (n f x)) (λf. λx. f x)

→ λf. λx. f ((λf. λx. f x) f x)

→ λf. λx. f (f x)  = 2
```

---

13. List Comprehension

Python - Sintaxă Nativă

```python
# Sintaxă: [expresie for item in lista if conditie]


numbers = [1, 2, 3, 4, 5]


# 1. Simple - Squares

squares = [x**2 for x in numbers]

# Rezultat: [1, 4, 9, 16, 25]


# 2. Cu filtrare - Evens

evens = [x for x in numbers if x % 2 == 0]
```

```python
# Rezultat: [2, 4]


# 3. Combinat - Even squares
even_squares = [x**2 for x in numbers if x % 2 == 0]
# Rezultat: [4, 16]


# 4. Cu transformare
words = ["hello", "world", "python"]
uppercase = [w.upper() for w in words]
# Rezultat: ['HELLO', 'WORLD', 'PYTHON']


# 5. Nested comprehension (matrice)
matrix = [[i*j for j in range(1, 4)] for i in range(1, 4)]
# Rezultat: [[1, 2, 3], [2, 4, 6], [3, 6, 9]]


# 6. Multiple conditii
filtered = [x for x in range(20) if x % 2 == 0 if x > 10]
# Rezultat: [12, 14, 16, 18]


# 7. If-else în expresie
result = ["Even" if x % 2 == 0 else "Odd" for x in range(5)]
# Rezultat: ['Even', 'Odd', 'Even', 'Odd', 'Even']


# 8. Flatten nested list
nested = [[1, 2], [3, 4], [5, 6]]
```

```
flat = [item for sublist in nested for item in sublist]

# Rezultat: [1, 2, 3, 4, 5, 6]


# 9. Dictionary comprehension

square_dict = {x: x**2 for x in range(5)}

# Rezultat: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}


# 10. Set comprehension

unique_squares = {x**2 for x in [-2, -1, 0, 1, 2]}

# Rezultat: {0, 1, 4}
```

C# - LINQ Equivalent

```
var numbers = new List<int> { 1, 2, 3, 4, 5 };


// 1. Sintaxă declarativă (Query Syntax)

var squares = from n in numbers

              select n * n;


var evens = from n in numbers

            where n % 2 == 0

            select n;


var evenSquares = from n in numbers

                  where n % 2 == 0
```

```
                    select n * n;


// 2. Sintaxă metodă (Method Syntax) - mai aproape de Python
var squares2 = numbers.Select(x => x * x);
var evens2 = numbers.Where(x => x % 2 == 0);
var evenSquares2 = numbers.Where(x => x % 2 == 0)
                          .Select(x => x * x);


// 3. Nested - Matrix
var matrix = Enumerable.Range(1, 3)
    .Select(i => Enumerable.Range(1, 3)
        .Select(j => i * j)
        .ToList())
    .ToList();


// 4. Multiple conditions
var filtered = Enumerable.Range(0, 20)
    .Where(x => x % 2 == 0)
    .Where(x => x > 10)
    .ToList();


// 5. If-else în select
var result = Enumerable.Range(0, 5)
    .Select(x => x % 2 == 0 ? "Even" : "Odd")
    .ToList();
```

```
// 6. Flatten
var nested = new List<List<int>> {
    new List<int> {1, 2},
    new List<int> {3, 4},
    new List<int> {5, 6}
};
var flat = nested.SelectMany(x => x).ToList();
// Rezultat: [1, 2, 3, 4, 5, 6]


// 7. Dictionary
var squareDict = Enumerable.Range(0, 5)
    .ToDictionary(x => x, x => x * x);


// 8. Distinct (Set-like)
var uniqueSquares = new[] { -2, -1, 0, 1, 2 }
    .Select(x => x * x)
    .Distinct()
    .ToList();
```

JavaScript - Array Methods

```
const numbers = [1, 2, 3, 4, 5];


// 1. Map (transformare)
const squares = numbers.map(x => x ** 2);
```

```
// [1, 4, 9, 16, 25]


// 2. Filter (filtrare)
const evens = numbers.filter(x => x % 2 === 0);
// [2, 4]


// 3. Map + Filter
const evenSquares = numbers
    .filter(x => x % 2 === 0)
    .map(x => x ** 2);
// [4, 16]


// 4. Nested (matrix)
const matrix = Array.from({length: 3}, (_, i) =>
    Array.from({length: 3}, (_, j) => (i+1) * (j+1))
);
// [[1,2,3], [2,4,6], [3,6,9]]


// 5. Flatten
const nested = [[1, 2], [3, 4], [5, 6]];
const flat = nested.flat();
// [1, 2, 3, 4, 5, 6]


// Sau cu flatMap:
const flat2 = nested.flatMap(x => x);
```

```
// 6. Reduce (aggregation)
const sum = numbers.reduce((acc, x) => acc + x, 0);
// 15


// 7. Object from array
const squareObj = Object.fromEntries(
    numbers.map(x => [x, x ** 2])
);
// {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Comparație Comprehension

| Operație | Python | C# LINQ | JavaScript |

|---------|--------|---------|----------|

| Transform | `[x*2 for x in list]` | `.Select(x => x*2)` | `.map(x => x*2)` |

| Filter | `[x for x in list if x > 5]` | `.Where(x => x > 5)` | `.filter(x => x > 5)` |

| Both | `[x*2 for x in list if x > 5]` | `.Where(...).Select(...)` | `.filter(...).map(...)` |

---

14. Memory Efficiency & Data Structures

Array vs List

```
// ARRAY - Fixed size, memory contiguă
int[] array = new int[5]; // Mărime fixă
array[0] = 10;
array[4] = 50;
```

```
// array[5] = 60; // IndexOutOfRangeException


// Caracteristici:
// - Mărime fixă, alocată la început
// - Memorie contiguă
// - Access rapid O(1)
// - Insertion/Deletion costisitoare


// LIST - Dynamic size
List<int> list = new List<int>();
list.Add(10);    // Adaugă la sfârșit
list.Add(20);
list.Remove(10); // Șterge primul 10
list.Insert(1, 15); // Inserează pe poziția 1


// Caracteristici:
// - Mărime dinamică, crește automat
// - Array intern (realocări când crește)
// - Access O(1), Insert/Delete O(n)
// - Metode utile: Add, Remove, Contains, Sort, etc.
```

Struct vs Class

```
// STRUCT - Value type (stack allocation)
public struct Point {
```

```
    public int X;
    public int Y;
    public Point(int x, int y) {
        X = x;
        Y = y;
    }
}

// Se copiază prin valoare
Point p1 = new Point(1, 2);
Point p2 = p1;  // COPIE completă
p2.X = 10;
Console.WriteLine(p1.X);  // 1 (nemodificat)
Console.WriteLine(p2.X);  // 10

// CLASS - Reference type (heap allocation)
public class Rectangle {
    public int Width;
    public int Height;
    public Rectangle(int w, int h) {
        Width = w;
        Height = h;
    }
}
```

```
// Se copiază referința

Rectangle r1 = new Rectangle(10, 20);

Rectangle r2 = r1;  // Ambele referă același obiect

r2.Width = 100;

Console.WriteLine(r1.Width);  // 100 (modificat!)

Console.WriteLine(r2.Width);  // 100
```

Când folosești Struct vs Class

Folosește STRUCT când:

- Obiect mic (< 16 bytes recomandat)
- Valoare imutabilă (nu se modifică după creare)
- Nu necesită inheritance
- Exemplu: Point, Color, Complex number

Folosește CLASS când:

- Obiect mare
- Necesită modificări
- Necesită inheritance/polymorphism
- Exemplu: Person, Car, Database

Passing Parameters

```
// BY VALUE (default pentru value types)

void ModifyValue(int x) {

    x = 100;  // Modifică doar copia

}



int a = 5;

ModifyValue(a);

Console.WriteLine(a);  // 5 (nemodificat)
```

```
// BY REFERENCE (cu ref)
void ModifyRef(ref int x) {
    x = 100;  // Modifică variabila originală
}

int b = 5;
ModifyRef(ref b);
Console.WriteLine(b);  // 100 (modificat)

// OUT parameter
void GetValues(out int x, out int y) {
    x = 10;
    y = 20;
}

int m, n;
GetValues(out m, out n);
Console.WriteLine($"{m}, {n}");  // 10, 20

// Reference types (clase)
void ModifyObject(Rectangle rect) {
    rect.Width = 200;  // Modifică obiectul original
}
```

```
Rectangle r = new Rectangle(10, 20);

ModifyObject(r);

Console.WriteLine(r.Width);  // 200 (modificat)
```

Collections - Complexitate

| Operație | Array | List | LinkedList | Dictionary | HashSet |

|---------|-------|------|----------|----------|---------|

| Access by index | O(1) | O(1) | O(n) | N/A | N/A |

| Search | O(n) | O(n) | O(n) | O(1) | O(1) |

| Insert at end | N/A | O(1)* | O(1) | O(1) | O(1) |

| Insert at start | N/A | O(n) | O(1) | N/A | N/A |

| Delete | N/A | O(n) | O(1)** | O(1) | O(1) |

*O(1) amortized (uneori O(n) când realocă)

**O(1) dacă ai referință la nod

---

15. Rezumat Final și Sfaturi pentru Examen

Checklist Subiecte

✅ **1. Piece of Code Analysis**

- AST construction
- Intermediate code
- Optimization
- Final assembly

✅ **2. Client-Server & Threading**

- Server multi-threaded
- Race conditions
- Lock/Mutex pentru thread safety

✅ **3. AST Construction**

- Operatori cu prioritate
- Arbori pentru if/while/for

✅ **4. Grammar & Semantics**

- Evaluarea expresiilor
- Action semantics
- Prioritate operatori

✅ **5. High-Level → Low-Level**

- IF/ELSE → JMP labels
- WHILE → LOOP labels
- FOR → Initialization + Loop

✅ **6. Weakest Precondition**

- wp(x = E, Q) = Q[x/E]
- Calculare de la coadă la cap
- Transmitere prin referință vs valoare

✅ **7. Compilation Phases**

- Lexical → Syntax → Semantic → IR → Optimize → Code Gen

✅ **8. Type Systems**

- Static vs Dynamic
- Strong vs Weak
- Comparație C/C++/Java/C#/Python/JS

✅ **9. OOP**

- Inheritance, Polymorphism, Encapsulation
- Abstract classes, virtual methods
- Hierarchy implementation

✅ **10. Design Patterns**

- Singleton
- Factory Method
- Observer

✅ **11. Exceptions**

- Throw custom exceptions
- Try-catch-finally

- Multiple catch blocks

✅ **12. Lambda Calculus**

- β-reduction
- Aplicare funcții
- Church numerals (posibil)

✅ **13. List Comprehension**

- Python: `[x for x in list if cond]`
- C#: `.Where().Select()`
- JS: `.filter().map()`

✅ **14. Data Structures**

- Array vs List
- Struct vs Class
- Pass by value vs reference

Strategii pentru Examen

1. Citește problema complet

- Identifică ce se cere exact
- Vezi dacă sunt exemple date

2. Porn de la exemple

- Dacă îți dă cod, urmărește execuția pas cu pas
- Scrie valori intermediate

3. Pentru AST

- Desenează arborele complet
- Operatori cu prioritate înaltă mai jos

4. Pentru Weakest Precondition

- Lucrează de la COADĂ la CAP
- Înlocuiește variabila cu expresia

5. Pentru traducere Low-Level

- Desenează flow-ul cu labels
- JMP pentru salt necondiționat
- JE/JNE/JG/JL pentru salturi condiționate

6. Pentru OOP

- Începe cu clasa de bază abstractă
- protected pentru proprietăți moștenite
- override pentru metode abstracte

7. Pentru LINQ

- Sintaxă: `from x in collection where ... select ...`
- Materializează cu `.ToList()` sau `.FirstOrDefault()`

8. Pentru Lambda Calculus

- Lucrează de la STÂNGA la DREAPTA
- Aplică funcția pe primul argument
- Continuă cu următorul

Greșeli Comune de Evitat

❌ Uitare `.ToList()` în LINQ

❌ Confuzie între `==` și `equals` în join

❌ AST cu prioritate greșită

❌ WP calculat de la cap la coadă (GREȘIT!)

❌ Uitare `lock()` în threading

❌ Confuzie struct vs class (value vs reference)

❌ Labels lipsă în traducere low-level

❌ Uitare `override` în metode derivate

Template-uri Rapide

OOP Hierarchy:

```
public abstract class Base {

    public string Prop { get; protected set; }

    protected Base(string prop) { Prop = prop; }

    public abstract void Method();

}
```

```
public class Derived : Base {

    public Derived(string prop) : base(prop) { }

    public override void Method() { /* ... */ }

}
```

Singleton:

```
private static T _instance;

private static object _lock = new object();

private T() { }

public static T Instance {

    get { lock(_lock) {

        if (_instance == null) _instance = new T();

        return _instance;

    }}

}
```

Thread-safe:

```
private object _lock = new object();

public void Method() {

    lock(_lock) {

        // Zona critică

    }
```

```
}
```

Resurse Finale

Pentru practice:

- Exerciții AST: Desenează pentru diverse expresii
- WP: Calculează pentru cod simplu → complex
- LINQ: Scrie query-uri pentru diverse scenarii
- Lambda: Reduce expresii simple → complexe

Timp estimat per secțiune (exemplu):

- AST: 5-10 min
- WP: 10-15 min
- Compilation phases: 5-10 min
- OOP hierarchy: 15-20 min
- LINQ queries: 10-15 min
- Lambda reduction: 10-15 min

Prioritizare pe baza punctajului din examen!

Succes la examen! 🚀

Ai acoperit toate conceptele fundamentale. Repetă secțiunile unde te simți mai slab și fă exerciții practice pentru fiecare tip de problemă.